Using a Photoresistor to Operate an LED

Purpose: To include a photoresistor that uses light detection to operate an LED.

Materials: Computer, Arduino Uno Board, USB cable, Breadboard, 1 LED, 1 - 220 Ω Resistor, 1 - 10KΩ resistor, 7 Jumper Wires, 1 Photoresistor

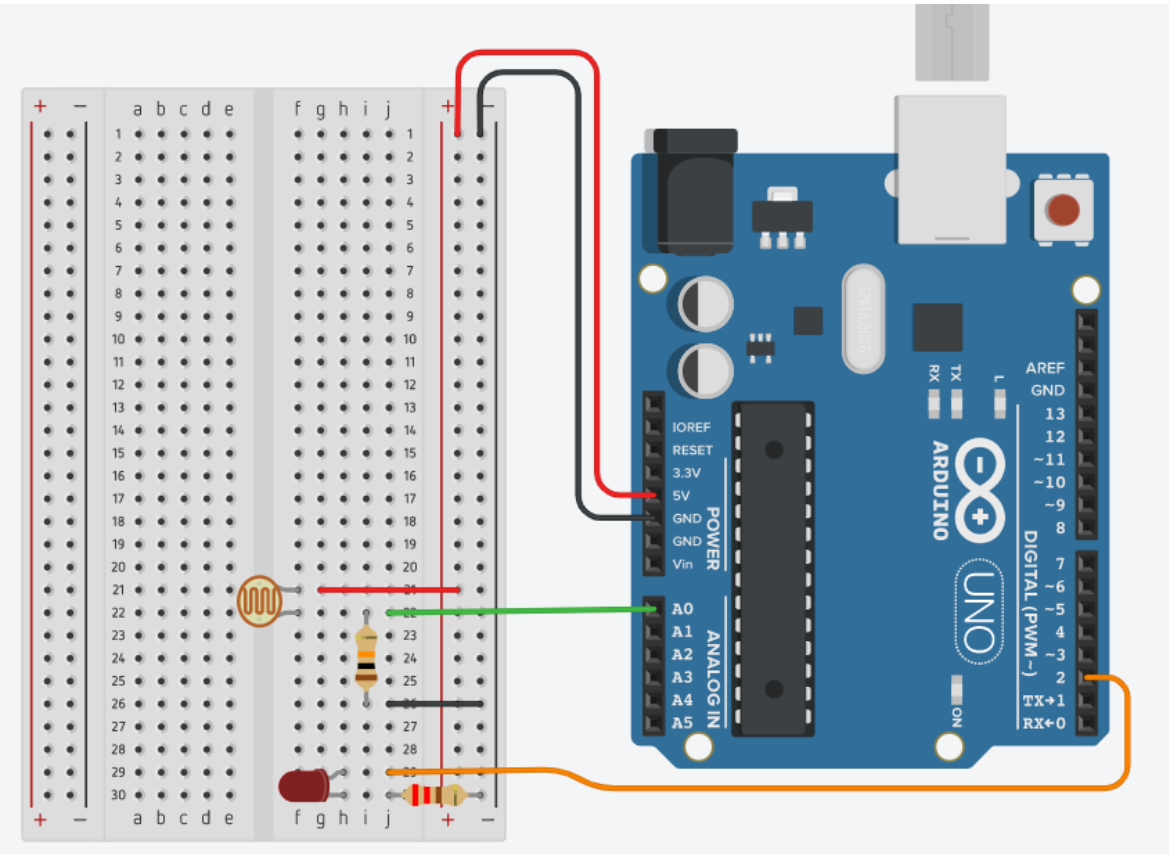
New Components:

Photoresistor. Photoresistors are light sensitive, variable resistors. As more light shines on the sensor's head, the resistance between its two terminals decreases. Therefore photoresistors can act as an switch that uses light detection to turn loads (such as light) on and off.



Wiring Diagram:

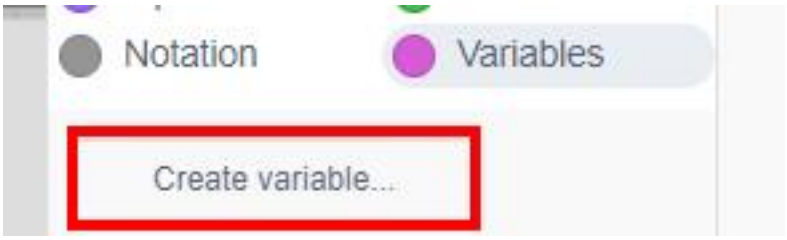
Wire the project using the following schematic.



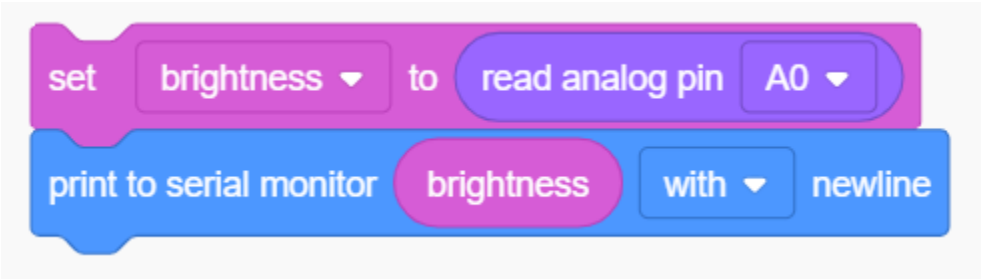
Initial Coding:

For this project, we want to be able to use a variable to store the amount of light shining on the Photoresistor.

Variables are used to store information to be referenced and manipulated in a computer program. They also provide a way of labeling data with a descriptive name, so our programs can be understood more clearly by the reader and ourselves. It is helpful to think of variables as containers that hold information. Their sole purpose is to label and store data in memory. This data can then be used throughout your program. We will create the integer variable “brightness”.



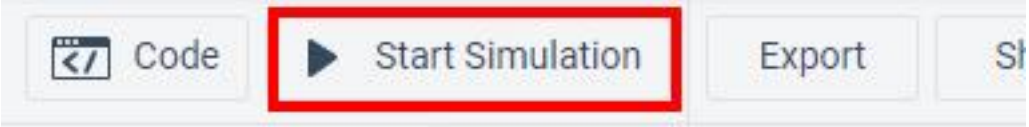
once you’ve created this variable, let’s try storing a value in it, and then print that value to the Serial Monitor (so we can see what its contents are)



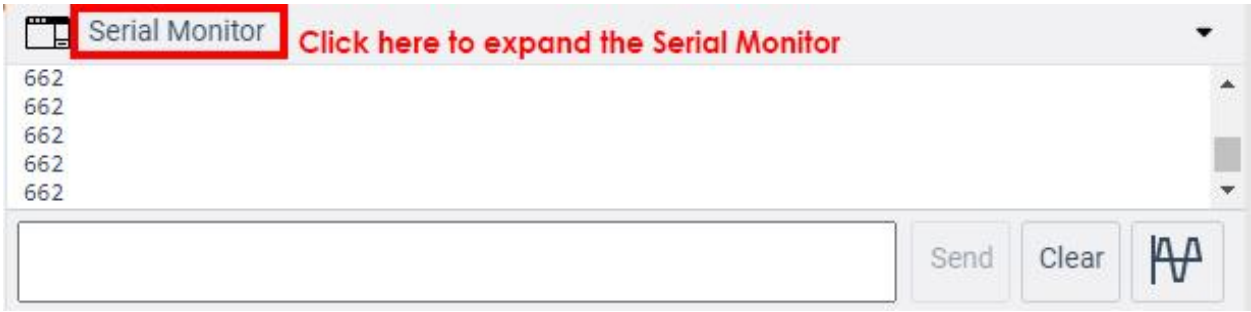
The first line reads the voltage on **analog pin A0**, which will be reported as a number between 0 (low voltage) and 1023 (high voltage). *Note that this is a different range than when we work with digital pins.*

The second line prints this number to the Serial Monitor and add a newline character so that there will be *one value printed per line.*

Press “Start Simulation” to begin running the program.

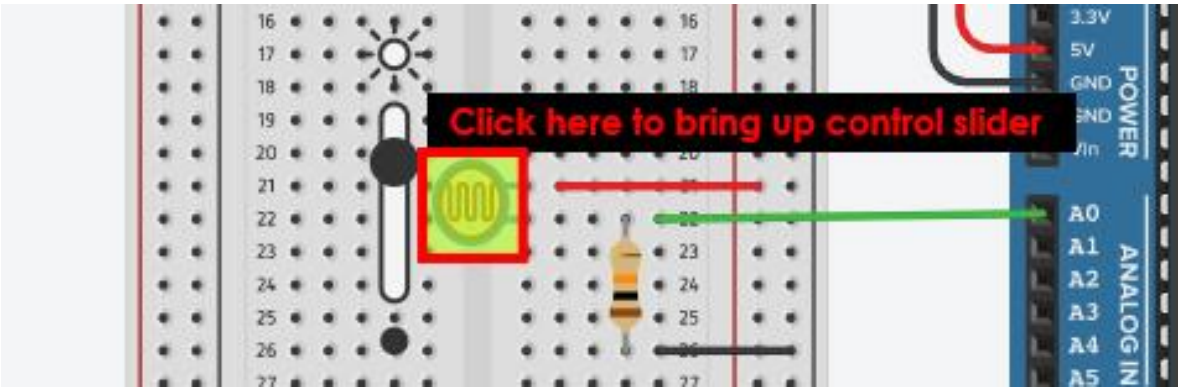


While it is running, you'll need to open the Serial Monitor to be able to view our printed values.



You should see a single number being printed out over and over. (not necessarily the one shown above – this number gives an idea of how much light is reaching the photoresistor head).

To simulate different amounts of light, click on the photoresistor component (While the simulation is running). This will bring up a slider where you can simulate brighter or darker situations.



Note: When running a physical circuit, the values will depend largely on where you are and the amount of light in the vicinity, so it may be useful to run the above program first to test what types of values we get in the room before progressing further.

Create Initial Physical Circuit and Test

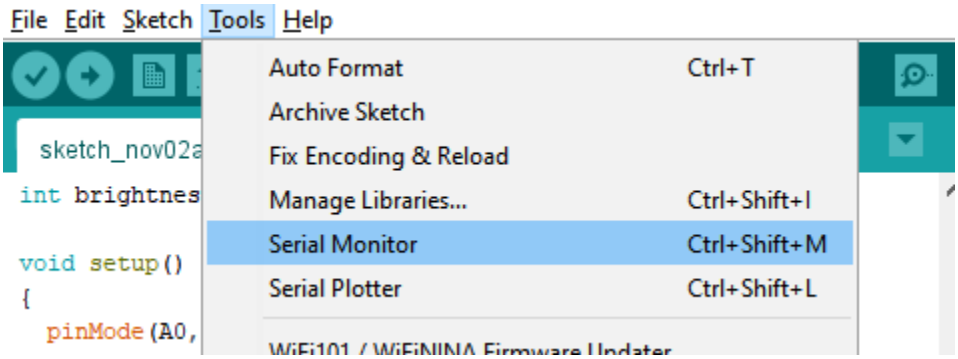
Take a moment to wire up a physical version of our simulated circuit, as we want to test the types of values we can get from the photoresistor in the room we are in.

In order to flash our program onto an Arduino, we need to get our code into the Arduino IDE. To do this, click on the **blocks** dropdown menu in TinkerCAD and choose **blocks+text**.



At this point you should be able to copy the code in the rightmost panel, and paste it into the Arduino editor.

Upload the program to your Arduino (through the Arduino IDE), and then open up the Serial Monitor to view the information we are getting from the photoresistor:



Find your lowest and highest values.

Experiment by shading the photoresistor with an open hand. If you bring the hand closer or farther from the component you should see our brightness value will change. We are interested in two values in particular:

Highest Value _____ *(not shading the photoresistor at all)*

Lowest Value _____ *(hand very close or touching photoresistor)*

Record these two values. An easy and convenient way to do this would be to place the information *into our program as a comment.* Any characters on a line that follow a ***//*** is ignored by the computer, so it simply a note for humans to read:

```
int brightness = 0;
//Highest Value: 975
//Lowest Value:  302
void setup()
```

Add Control for the LED

Now that we have an idea of what values we can expect from the photoresistor, let's try using it to control an LED. Our LED is wired into **digital pin 2** (*as an output pin*). For our initial program, let's use the following logic:

Based on our experimental high/low values, determine the **midpoint value.** Store this in a variable.

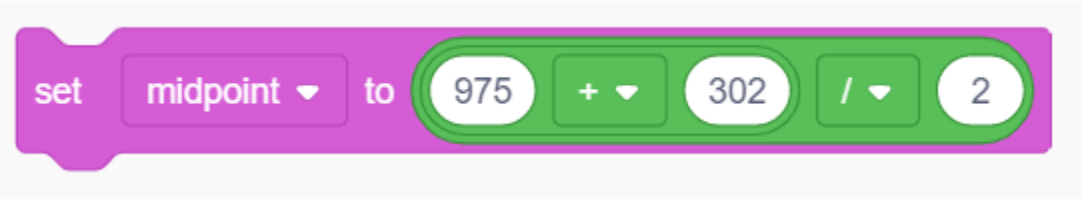
If the current **brightness** is larger than the **midpoint value**: turn the **LED on**

If the current **brightness** is equal to our less than the **midpoint value**: turn **LED off**

> **Try it:** *If you think you can code the algorithm above on your own, please do so before looking further in this document.*
>
> *If you need help or are ready to compare your work with the provided solution, continue to scroll down to see the code.*

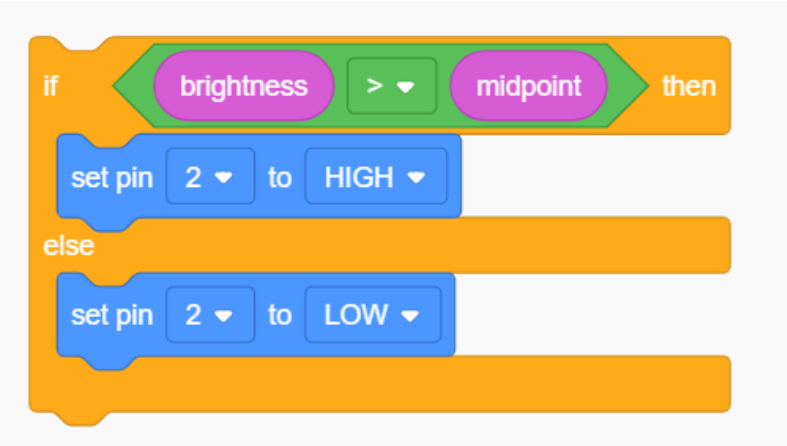
LED CODE

Start by creating a new variable (midpoint) to store your middle brightness value (based on your earlier experimentation).



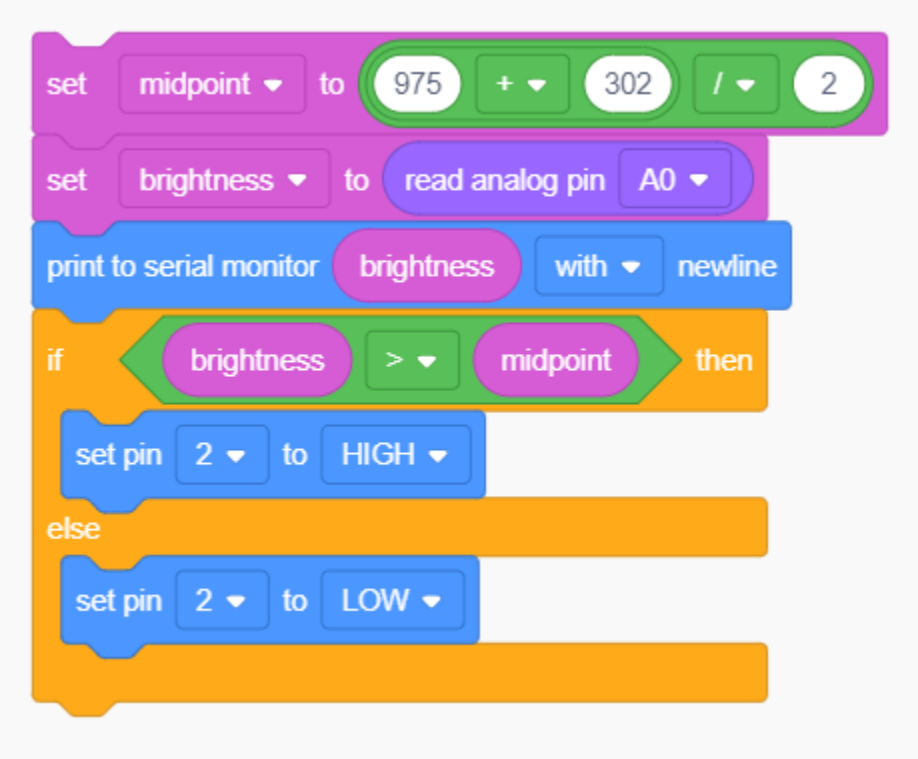
High and low value added together, then divided by two to get average or midpoint value.

- To add the IF/ELSE logic, look in the **Control Block Palette.**
- You'll also need Comparison operators (greater than, less than, etc…), which can be found in the **Math Block Palette.**
- Lastly, you'll need the variable data blocks which are in the **Variables Block Palette.**



Test in Simulator, then with Physical Circuit.

Combine the new blocks with the existing code we had. Make sure that the logic comparison happens *after* the brightness variable is updated. Your finished program may look like this:



Try this first in the simulator. You should be able to use the brightness value as a switch to turn the LED on or off.

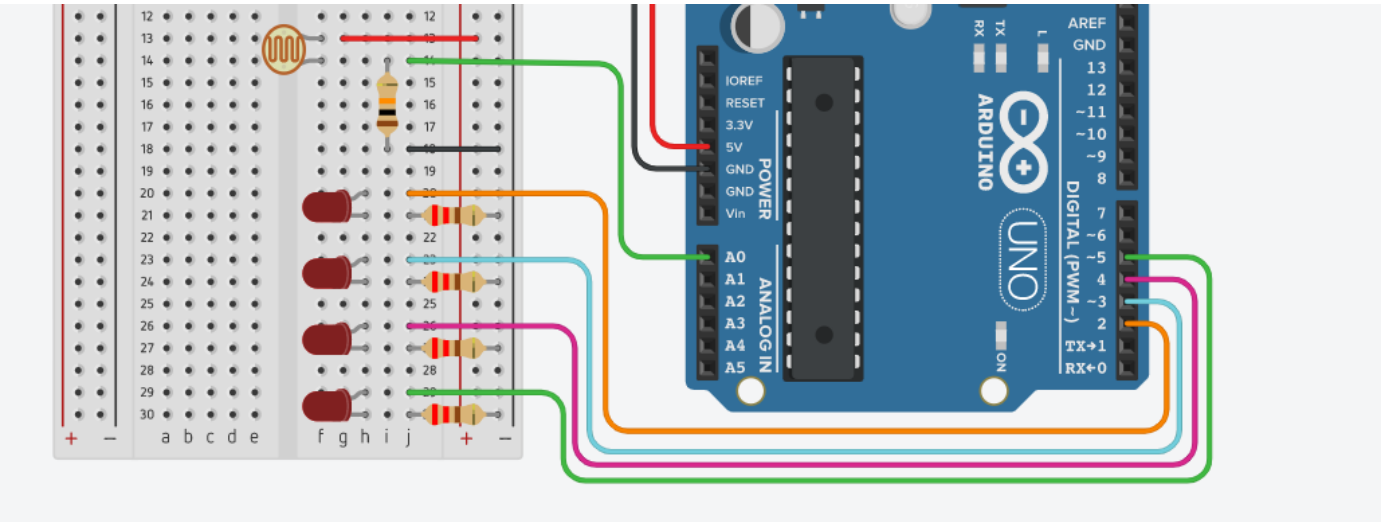
Once you have that working, copy your **text code** (like we did earlier) into the Arduino Editor and test it out with the physical circuit.

Extension

Multi-Power LED nightlight

One common application of a photoresistor is to have a light activate when it becomes dark. This could be street lights, or perhaps a night light. It would be possible to make a multi-LED system where *the darker it is, the more LED lights will be on.*

Expand your circuit to include 3-4 additional LEDs, something like the following:



A few things to remember when expanded your code:

- Each LED is connected to a different digital pin (2, 3, 4, and 5), so you'll need to set each to HIGH or LOW, depending on the brightness value.
- You may want to use the following logic:
  - If the brightness is very low:
    set all LED pins to LOW (0)
  - If the brightness is pretty low (choose a value based on your low/high range):
    set LED (pin 5) to HIGH
    set the rest of the LEDS to LOW
  - And so on…

Once complete – test your circuit to see if the number of illuminated LEDs can be controlled by how much you shade the photoresistor.